



Cerberus: Cross-Layer ECC Co-Design for Robust and Efficient Memory Protection

Junhwan Kim^{*¶}, Seunghyun Kim^{†¶}, Yesin Ryu[‡], Saeid Gorgin[§], and Jungrae Kim^{*}

^{*}Dept. of Electrical and Computer Engineering, [†]Dept. of Artificial Intelligence,

[‡]Dept. of Semiconductor and Display Engineering, Sungkyunkwan University, Suwon, Republic of Korea

[§]School of Physics, Engineering and Computer Science, University of Hertfordshire, Hatfield, United Kingdom
{june9918, kshyun0815, seleneyou}@skku.edu, s.gorgin@herts.ac.uk, dale40@skku.edu

Abstract—As DRAM scales to higher density and I/O speeds, ensuring data correctness becomes increasingly difficult. Industry has responded with a three-layer stack—on-die ECC (O-ECC), link ECC (L-ECC), and system ECC (S-ECC)—but these layers have evolved independently, often duplicating redundancy, leaving coverage gaps, and occasionally interfering.

We propose *Cerberus*, a cross-layer ECC co-design that unifies protection across device, link, and system while preserving each layer’s native role. At its core is an *Encode-Once, Decode-Many (EODM)* architecture: the controller performs a single encoding whose redundancy is reused by L-ECC for immediate write-path detection/retry, by O-ECC for in-device repair on reads, and by S-ECC for strong end-to-end recovery. Cerberus jointly designs complementary parity/syndrome structures, orders decoders and allocates the correction budget to prevent miscorrection amplification, and enables selective correction under tight redundancy constraints. Our evaluations show improved resilience to clustered and peripheral faults while reducing redundancy overhead, underscoring the importance of coordinated cross-layer protection for next-generation memory systems, such as custom HBMs.

Index Terms—DRAM, Reliability, ECC, Multi-layer ECC

I. INTRODUCTION

As memory technology scales toward ever higher density and bandwidth, ensuring data correctness has become increasingly challenging [1]–[4]. Modern DRAMs integrate billions of tiny capacitors that store femtocoulomb-level charge, operate at sub-nanosecond timing margins, and communicate over multi-gigabit-per-pin I/O links [5]–[8]. These advances enable unprecedented capacity and performance but also amplify vulnerability to faults. A single defective transistor, a marginal sense amplifier, or a transient signaling glitch can corrupt data and compromise large-scale computation [9], [10].

The risk is especially pronounced in recent DRAM families such as HBM and LPDDR. These memories map each channel to a single DRAM device (a *single-device-per-channel (SDPC)* organization) to achieve higher bandwidth and lower power. However, this decision places all data in a single device, effectively putting all eggs in a single basket. Consequently, any device-level defect can compromise the entire channel, whereas traditional multi-device DDR modules can tolerate a failed device through rank-level protection [11].

At the same time, modern data-intensive workloads have reduced the minimum fetch granularity of SDPC memories

to 32B (vs. 64B in DDR-based systems) to curb overfetch and improve effective bandwidth. While beneficial for performance, the smaller granularity leaves less room to amortize ECC overheads. As a result, SDPC memories typically allocate only 2 B of redundancy per 32 B block (6.25%), versus 8 B per 64 B (12.5%) in common DDR configurations. This narrow budget leaves little margin for robust protection [12], [13].

Historically, DRAM reliability relied on a *single layer of Error-Correcting Code (ECC)* at the memory controller—*System ECC (S-ECC)*—with simple *Single-Error Correction, Double-Error Detection (SEC-DED)* sufficient when isolated soft errors dominated [14]. However, as process, voltage, and I/O scaling approach physical limits, fault modes have shifted [15]–[17]. Variability, device aging, and high-speed signaling induce *clustered* and *peripheral* faults spanning 8–32 bits within a single access, alongside transient link corruptions.

These multi-bit errors exceed SEC-DED’s correction capability and can occasionally evade detection, leading to *Silent Data Corruption (SDC)* that threatens end-to-end correctness. Beyond the technical implications, these trends also affect manufacturability and product quality. Achieving defect-free DRAM dies has become increasingly difficult as geometries shrink, leading to declining yield. Worse, marginal and intermittent defects often elude production testing, escaping into the field as latent faults [18].

To mitigate both reliability and manufacturability challenges, the DRAM industry has incrementally introduced protection mechanisms at *multiple layers* of the memory hierarchy [19]. Modern devices now integrate *On-die ECC (O-ECC)* to locally repair manufacturing defects and small-scale faults, thereby improving yield [20]–[23]. At the channel boundary, high-speed interfaces employ *Link ECC (L-ECC)* or *Cyclic Redundancy Check (CRC)* to detect transmission errors in real time [24]–[26]. Meanwhile, memory controllers continue to enhance *S-ECC* to recover from large-granularity failures, such as a dead chip within a module [27]. Collectively, these mechanisms form a *three-layer ECC hierarchy*—link, on-die, and system—that protects data across distinct physical and temporal domains.

However, these layers have evolved independently, without cross-layer coordination. As a result, they often duplicate redundancy, leave critical fault regions unprotected, and most critically suffer from destructive interference across layers. For

[¶]Both authors contributed equally to this research.

example, when errors exceed the correction capability of O-ECC, the decoder may *miscorrect* benign bits, increase the number of corrupted bits and turn a pattern S-ECC could have fixed into one it cannot [28]. L-ECC prevents write-path corruption but operates orthogonally to both O-ECC and S-ECC, limiting end-to-end fault traceability. This fragmented protection model wastes redundancy and struggles to balance reliability, performance, and cost in modern memory systems.

This paper introduces *Cerberus*, a novel cross-layer ECC co-design that unifies protection across the link, device, and system layers. Cerberus addresses the challenges of current multi-layered memory systems, where O-ECC, L-ECC, and S-ECC are provisioned and managed in isolation. Instead of treating them as separate features, Cerberus co-optimizes their roles so that their checks become complementary rather than redundant or destructive, while preserving each layer’s native responsibilities: O-ECC for local repair and yield improvement, L-ECC for real-time detection and retransmission, and S-ECC for robust end-to-end recovery.

At the heart of Cerberus is an *Encode-Once, Decode-Many* (EODM) architecture. On writes, the memory controller performs a single encoding step to generate a shared redundancy for all layers. Along the write path, the link layer utilizes the redundancy to detect transmission errors immediately and trigger retransmission when needed (write-side L-ECC). On reads, a bank-group decoder corrects storage-side faults within the device (O-ECC) using that same redundancy. Finally, the controller consumes the full redundancy to deliver strong end-to-end correction and detection against cell, peripheral, and I/O failures (S-ECC and read-side L-ECC). By reusing a single pool of redundancy across layers, EODM improves efficiency and provides seamless coverage without protection gaps observed in HBM configurations.

Realizing this architecture requires novel ECC codes that enable redundancy reuse across layers. The limited redundancy must be allocated across layers to balance their costs and benefits, and reuse must be orchestrated so that any mis-correction at an earlier layer never amplifies the error burden passed to later layers. The scheme must also be sufficiently scalable to support practical co-design and deployment across processor and DRAM vendors.

Cerberus achieves these goals by co-designing the generator and parity-check matrices used for ECC. This matrix-level co-design enables interoperability across layers while allowing each layer to fully meet its own protection goals. We also enforce a bounded-fault constraint on O-ECC to ensure that any mis-correction never amplifies the error burden to the system layer. Furthermore, by varying the redundancy within the range feasible for SDPC, Cerberus serves as a scalable framework that can be applied across diverse memory systems.

Our evaluation shows that Cerberus improves reliability and performance despite lowering HBM’s storage overhead by 33.3%. With less redundancy than baseline HBM, Cerberus still achieves higher overall reliability, and under the same redundancy budget as LPDDR6, it delivers substantially stronger protection across diverse fault locations. In addition,

Cerberus improves system performance by 0.7%, mainly by eliminating consecutive encoder stages. Taken together, these results demonstrate that Cerberus is a practical and scalable framework for enhancing reliability and performance in future SDPC-enabled systems.

The major contributions of this paper are as follows:

- We propose the co-design architecture of Cerberus based on the EODM structure. By co-designing unified protection across each memory layer, Cerberus ensures consistent high reliability across the system while minimizing redundancy.
- We propose an HBM architecture that applies Cerberus. Cerberus achieves both system reliability and efficiency by significantly reducing parity storage overhead while maintaining high system-wide reliability against clustered and peripheral faults.
- We evaluate Cerberus using various workloads, showing that Cerberus achieves high performance and lower energy consumption while maintaining superior overall reliability.

II. BACKGROUND

This section reviews the fundamentals of ECC and how modern memory systems deploy them across *three layers*. It then summarizes recent observations on DRAM error characteristics that motivate cross-layer ECC designs.

A. Error Correcting Codes

Error Correcting Codes (ECC) enable reliable storage and transmission in the presence of physical faults. Most memory systems employ *linear block codes* for their low latency and hardware simplicity [29]. An (n, k) linear block code maps a k -symbol message $\mathbf{m}$ into an n -symbol *codeword* $\mathbf{c}$ by appending $r = n - k$ redundancy symbols [30]. A code can be described by a *generator matrix* G with $\mathbf{c} = \mathbf{m}G$ and a *parity-check matrix* H with $H\mathbf{c}^T = \mathbf{0}$.

Upon receiving $\mathbf{y} = \mathbf{c} + \mathbf{e}$ (where $\mathbf{e}$ is the error vector), the decoder computes the *syndrome* $\mathbf{s} = H\mathbf{y}^T = H\mathbf{e}^T$ [31]. Because $\mathbf{s}$ depends only on the error pattern, the decoder estimates $\hat{\mathbf{e}}$ and reconstructs $\hat{\mathbf{c}} = \mathbf{y} - \hat{\mathbf{e}}$. Decoding outcomes are commonly categorized as: (1) *Detectable and Correctable Error (DCE)*; (2) *Detected but Uncorrectable Error (DUE)* when $\mathbf{s}$ lies beyond the correction radius; (3) *Detected but Mis-corrected Error (DME)*, in which the decoder asserts success yet outputs an incorrect codeword; and (4) *Undetectable and Uncorrectable Error (UUE)* with $\mathbf{s} = \mathbf{0}$ despite corruption [32]. Cases (3) and (4) lead to *Silent Data Corruption (SDC)*, which is the most severe class of memory reliability failure [33]–[35].

The most common main-memory code is *Single-Error Correction and Double-Error Detection (SEC-DED)* [36], [37]. An SEC-DED code guarantees correction of any single-bit error and detection (but not correction) of any double-bit error. More powerful BCH codes correct t bit errors in an n bit word with redundancy on the order of $t\lceil\log_2(n+1)\rceil$ bits [38].

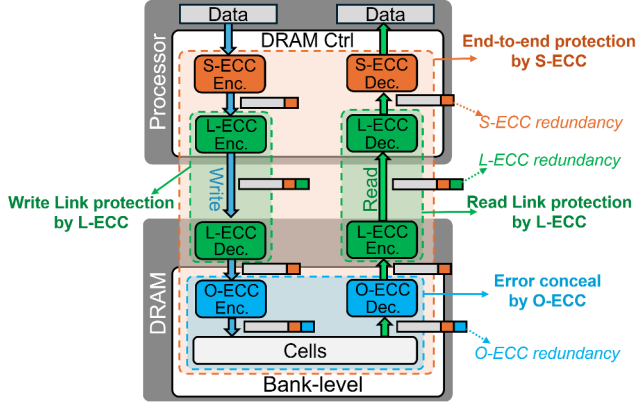


Fig. 1: Contemporary DRAM protection based on three ECC layers (S-ECC, O-ECC, and L-ECC)

Because DRAM errors frequently occur in bursts or are confined to specific physical regions, systems often employ *non-binary, symbol-based* codes. *Reed–Solomon (RS)* codes defined over $GF(2^m)$ can correct up to t symbol errors using $2t$ redundant symbols [39]. By aligning code symbols with physical fault domains—such as a chip or *Data Pin (DQ)*—RS codes efficiently convert spatially correlated bit errors into a small number of symbol errors, providing strong protection with modest redundancy [40]–[42].

B. System ECC

Processors have long employed ECC to ensure memory and overall system reliability [43], [44]. The memory controller encodes data on every write and decodes data on every read (orange in Fig. 1). This mechanism—known as *System ECC (S-ECC)* or *rank-level ECC*—detects and corrects errors in both storage and transmission, providing *end-to-end protection* between the processor and memory [45].

S-ECC implementations vary across DRAM types, generations, and processor vendors. A standard DDR4 ECC-DIMM provides a $(64+8)$ -bit interface, where the additional eight bits store redundancy for each 64-bit data beat. Most processors implement SEC-DED, enabling correction of any single-bit error and detection of double-bit errors per beat. While this configuration is sufficient for random soft errors, large-scale systems require stronger resilience against complete device failures—known as *Single-Device Data Correction (SDDC)* or *chipkill-correct* [46].

SDDC implementations vary across vendors and are often proprietary or confidential [47], [48]. Nonetheless, several public designs employ RS-based protection while remaining compatible with standard ECC-DIMMs. For instance, AMD’s *Chipkill-Correct* constructs 8-bit RS symbols by grouping two 4-bit data beats from each chip and employs two redundant devices for single-chip recovery [46]. *Bamboo ECC* forms 8-bit symbols across eight I/O beats, allowing correction of up to four faulty pins ($\approx$ one chip failure) [32]. More recently,

Unity ECC extends this concept to handle both single-symbol and double-bit errors through hybrid decoding [22].

DDR5 reshapes S-ECC design by doubling the burst length (from 8 to 16 beats) and dividing each DIMM into two 32-bit subchannels. High-reliability ECC-DIMMs allocate eight ECC pins per subchannel, forming $(2 \times (32 + 8))$ interfaces that provide SDDC capability within each subchannel [24]. In contrast, memory types such as HBM and LPDDR deliver full-channel access through a single device to maximize bandwidth and energy efficiency. In these *single-device memory channels*, device-level SDDC is infeasible—any device failure compromises the entire channel’s data. Consequently, S-ECC in such systems targets large-granularity burst or link-level faults rather than complete device failures.

C. On-Die ECC

Shrinking process technology reduces the charge stored in each cell, increases process variation, and makes transistors more susceptible to wear-out [3]. To mitigate these errors, modern DRAMs integrate *On-Die ECC (O-ECC)* that repairs faults internally, complementing external S-ECC [20], [49], [50]. Each die contains hidden redundancy cells and a compact encoder/decoder that encodes every write and decodes every read within each bank group (blue in Fig. 1). By correcting faults locally, O-ECC effectively converts dies with marginal defects into externally fault-free components, improving manufacturing yield and ensuring component-level reliability beyond warranty thresholds [21].

O-ECC implementations also vary across DRAM types and generations [51]–[54]. In DDR5, where channel data are distributed across multiple chips, devices employ lightweight bit-level SEC schemes with 8 redundant bits per 128 data bits [24]. In contrast, single-device memory channels such as HBM and LPDDR apply O-ECC at their native 32-byte access granularity. For instance, HBM4 uses 32 bits of redundancy to correct 16-bit-aligned symbol errors, while LPDDR6 applies SEC-DED codes with 16 bits of redundancy [25], [26].

D. Link ECC

As I/O speeds reach tens of gigabits per second per pin and signaling voltages continue to scale down, transient transmission errors have become a significant reliability concern [55]. While S-ECC provides end-to-end protection, it decodes data only on reads and therefore cannot detect *write-path* errors, which can leave corrupted data permanently stored in DRAM. Moreover, S-ECC is often omitted entirely in cost-sensitive or low-power systems to reduce pin count, area, and power consumption [56].

Link ECC (L-ECC) protects data during transmission between the memory controller and DRAM (green in Fig. 1). The sender (e.g., the controller on writes) encodes data before transmission, and the receiver verifies it immediately upon arrival, enabling rapid detection—and, in some designs, correction—of transient link errors. Upon error detection, the receiver can request retransmission, preventing corrupted data from being committed to DRAM. L-ECC thus serves as the

first line of defense in the data path, prioritizing fast detection and low-latency recovery over complex correction.

Different memory types adopt L-ECC in various forms. DDR5 employs an 8-bit *Cyclic Redundancy Check (CRC)* [57] per four DQs, while HBM implements a data-parity bit across every 32 DQs. LPDDR6 adopts 16-bit parity, configurable for either single-error correction or detection-only operation.

E. DRAM Errors

Designing efficient ECC mechanisms requires understanding how DRAM errors manifest in real systems. While individual DRAM chips are highly reliable, large-scale field studies reveal that aggregate errors exhibit non-negligible rates and distinct patterns [15], [16], [47], [58]–[61]. We summarize observations from recent studies as follows:

1) *Scaling-Induced Cell and Circuit Faults*: As DRAM technology continues to scale down, transient soft errors are increasingly overshadowed by permanent or intermittent faults caused by process variation and device wear-out. Individual cells have become more susceptible to charge leakage, variable retention time (VRT), and disturbance effects such as row hammering, while peripheral circuits suffer from degraded timing margins and transistor aging [1], [2], [18], [62].

2) *Multi-bit Errors*: Modern DRAMs increasingly exhibit *spatially correlated* multi-bit errors rather than isolated single-bit errors. Such correlations arise because many peripheral components—such as subwordline (SWL) and subwordline drivers (SWD)—serve multiple adjacent cells [16]. When one of these shared components fails, it can simultaneously corrupt all of the cells it serves. Column-related faults typically flip one bit per access, whereas row-related faults can disrupt multiple bits within the same access and thus pose a greater challenge to ECC [15].

The scope of these correlated errors depends heavily on the internal organization of peripheral circuits. Recent characterization of DDR5 devices reveals that most errors remain confined within a small physical region, typically spanning up to 16 bits per access [16]. In DDR5, each access transfers 8 bits of data from multiple *Memory Array Tiles (MATs)*. Although MATs are largely independent, adjacent tiles share critical peripheral components—most notably the subwordline driver. A defect in this shared driver can propagate across MAT boundaries, corrupting both tiles and resulting in up to 16 erroneous bits per access¹. This observation implies that modern ECC mechanisms must be capable of correcting up to 16 clustered errors to maintain high reliability in advanced DRAM technologies.

3) *Errors Beyond Bank-Groups*: While O-ECC effectively corrects faults within individual bank groups, recent studies reveal that a significant portion of DRAM errors originate beyond these boundaries, such as in device-level peripheral circuits or interconnect paths [58], [61], [63]. For example, a report on HBM3 devices equipped with integrated O-ECC revealed that, even with O-ECC enabled, a substantial number

of error interrupt events were still reported [60]. This implies that these errors originated outside the coverage of O-ECC or emerged after the O-ECC stage. The persistence of such errors indicates that many arise in unprotected regions—e.g., global I/O interfaces, TSV or silicon interposer links—where O-ECC’s correction scope does not apply [64]. These findings highlight the limitations of O-ECC and reinforce the importance of maintaining end-to-end protection through S-ECC.

III. MOTIVATION

The previous section outlined three ECC layers, each optimized for a distinct reliability objective: S-ECC provides end-to-end protection and strong overall reliability, O-ECC conceals errors and improves manufacturability [20], [21], [50], [65], and L-ECC enables early detection of link errors. This section examines how commercial DRAMs combine these layers in practice. Although each layer is effective in isolation, their ad-hoc integration frequently results in redundant coverage, inefficient use of redundancy, and—paradoxically—reduced overall reliability, motivating the need for a cross-layer ECC framework.

A. ECCs in DDR-based Systems

Although Cerberus targets single-device memory architectures such as HBM and LPDDR, it is instructive to first examine DDR-based systems. High-reliability platforms, including supercomputers, have developed sophisticated reliability mechanisms for DDR memory, and the distribution of data across multiple DRAM devices within a DIMM inherently enables strong SDDC protection [46].

A typical DDR5 system employs three ECC layers configured as follows: (1) *S-ECC*, implemented with 25% additional devices to provide SDDC-level protection; (2) *O-ECC*, adding 6.25% cell-area overhead (8 parity bits per 128-bit data word) for SEC within each device; and (3) *L-ECC*, introducing a 12.5% transfer overhead (two additional beats per 16-beat burst) to provide CRC16-based link error detection. Together, these layers incur approximately 32.8% storage overhead (from S-ECC and O-ECC) and 40.6% transfer overhead (from S-ECC and L-ECC), highlighting the inefficiency of independently managed ECC layers [24].

Despite these costs, reliability can degrade due to *miscorrections* [50], [66]. When two bits fail, an O-ECC configured as SEC may wrongly flip a third bit (Fig. 2a). If this new error falls in a different S-ECC symbol, the number of erroneous symbols may exceed S-ECC’s correction capability, producing an uncorrectable fault. Such miscorrections can occur at nontrivial rates. Under an SEC O-ECC + SEC-DED S-ECC stack, prior work reports that O-ECC miscorrects $\approx 45\%$ of double-bit errors (DBEs) into triple-bit errors, and that S-ECC then miscorrects these triple-bit errors as single-bit errors in $\approx 55\%$ of the cases, causing SDC [50]. It also estimates that SDC can occur once per 3 million accesses when the DRAM raw error rate is 10^{-4} .

To prevent such cross-layer interference, DDR5 enforces the *Bounded Fault (BF)* rule, which restricts each correction

¹Depending on the DRAM architecture, the affected bit width can range from about 8 to 32 bits.

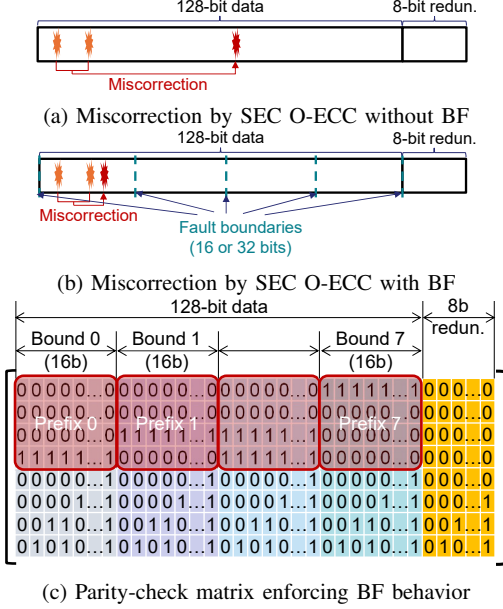


Fig. 2: Bounded-fault design for SEC O-ECC in DDR5

to a small spatial region (Fig. 2b)—typically 16 bits from an I/O pin. O-ECC must ensure that mis-corrections remain within the boundary region [28]. This behavior is guaranteed by designing the parity-check matrix H such that no sum of columns within a region equals any column outside that region. In one such parity-check matrix (Fig. 2c), columns within one region share a prefix: odd-bit errors preserve it (staying local), while even-bit sums cancel to zero, mapping to non-data space.

The BF rule effectively isolates O-ECC from S-ECC, allowing intra-device correction without propagating faults. However, each layer still maintains separate redundancy, inflating total storage overhead, and the BF layout constrains S-ECC’s symbol organization to Bamboo-ECC-like groupings [32].

B. ECCs in HBM-based Systems

HBM transfers data through a single device, allowing the granularity of O-ECC to align directly with that of S-ECC. In HBM4, each pseudo-channel protects 32 bytes of data with 2 bytes of S-ECC, 4 bytes of O-ECC, and 1 byte of L-ECC redundancy [26]. This configuration emphasizes on-die correction by allocating more redundancy bits to O-ECC, allowing it to correct up to 16 faulty bits per block. Such strong on-die protection effectively suppresses scaling-induced faults, especially those originating from peripheral circuits such as subwordline drivers. Meanwhile, HBM4 maintains high bandwidth by transmitting L-ECC through a dedicated sideband pin, leaving the main data interface fully utilized. To balance total redundancy, HBM4 limits S-ECC to 2 bytes per 32-byte data block. This limited budget can be used for either an ECC (e.g., SEC-DED) or an *Error Detecting Code* (EDC) (e.g., CRC16). In practice, most systems adopt CRC since its misdetection probability ($\approx 0.002\%$) is nearly two orders of magnitude lower than that of SEC-DED ($\approx 0.4\%$), substantially reducing the risk of SDC [52].

Despite these refinements, HBM’s reliability remains constrained by several structural limitations. First, because O-ECC performs symbol-based correction without a bounded-fault constraint, a mis-correction can produce errors beyond the correction capability of S-ECC, leading to an unrecoverable condition at the system level. Second, O-ECC can detect most uncorrectable events with high probability ($\approx 99.97\%$), but the results are conveyed to the system level only through a limited severity (SEV) pin [26], [67]. As a result, the system lacks sufficient cross-layer visibility, and S-ECC must rely solely on the limited error information interpreted by O-ECC (e.g., CE or UE signals), performing decoding under incomplete awareness. Consequently, the system becomes exposed to errors undetected by S-ECC, increasing the risk of missing multi-symbol errors. Third, O-ECC’s protection scope is confined to the internal array and peripheral circuitry; link and I/O interface errors remain outside its coverage. This issue is particularly severe in HBM, where Through-Silicon Vias (TSVs) introduce new fault modes. Because a TSV fault lies outside O-ECC’s protection scope, detection-based S-ECC alone cannot adequately handle this error. Recent field studies of HBM3 devices [60] corroborate these issues, showing that a substantial number of errors are still detected by S-ECC, indicating that many faults occur beyond the reach of O-ECC.

C. ECCs in LPDDR-based Systems

Similar to HBM, LPDDR employs a single-device memory channel architecture, allowing S-ECC and O-ECC to operate at the same granularity. In LPDDR6, each pseudo-channel protects a 32-byte data block using 2 bytes of S-ECC, 2 bytes of O-ECC, and 2 bytes of L-ECC redundancy. The O-ECC employs its 2-byte budget to implement SEC-DED for the 32-byte data block and its associated S-ECC redundancy. Externally, a subchannel transfers 36 bytes per burst across 12 I/O pins over 24 beats—32 bytes of user data, 2 bytes of S-ECC, and 2 bytes carrying either L-ECC redundancy or *Data Bus Inversion* (DBI) information. Overall, this configuration introduces about 12.5% storage and transfer overhead, a cost considered acceptable in mobile devices [25].

Despite these protections, LPDDR6 still provides only moderate reliability due to two key limitations: *mis-correction propagation* and *limited correction capability*. Because LPDDR6 lacks Bounded Fault (BF) protection, an O-ECC mis-correction can turn a triple-bit error into a quadruple-bit corruption, increasing the burden on S-ECC. In addition, SEC-DED-based O-ECC can correct only single-bit errors, so it faces a fundamental limitation when handling multi-bit errors that often arise in peripheral circuitry. The same constraint applies to S-ECC when it uses the same redundancy budget as O-ECC. If S-ECC is configured as SEC-DED, it also corrects only single-bit errors and cannot handle multi-bit errors. Alternatively, configuring S-ECC as an 8-bit Single Symbol Correction (SSC) allows correction of up to 8-bit clustered errors, but its detection capability is limited ($\approx 86.7\%$).

TABLE I: Comparison of ECC schemes

		Single-layer		Multi-layer				Cross-layer
		DUO	Unity ECC	LPDDR6/ SEC-DED	LPDDR6/ CRC	HBM4/ SEC-DED	HBM4/ CRC	Cerberus
Total	Data bits	512		256				
	Storage overhead	19.5% (100b)	25% (128b)	12.5% (32b)		18.8% (48b)		12.5% (32b)
	Transfer overhead	19.5% (100b)	25% (128b)	12.5% (32b)		15.6% (40b)		12.5% (32b)
S-ECC	Bit config.	512 + 100	512 + 128	256 + 16				256 + 16 + 16
	ECC	RS(76,64)	SSC+DEC	SEC-DED	CRC	SEC-DED	CRC	SSC+DEC
O-ECC	Bit config.	N/A		272 + 16		272 + 32		272 + 16
	ECC			SEC-DED		16b SSC		SEC-DED
L-ECC	Bit config.	N/A		272 + 16		272 + 8		272 + 16
	ECC			SEC or EDC		Parity		EDC
Early Detection		No		Yes				
Error Concealment		No		Yes				
Bounded Fault				No				Yes
Correction		High		Low		Medium	Low	High
Detection		Very High	High	Low	High	Medium	High	High

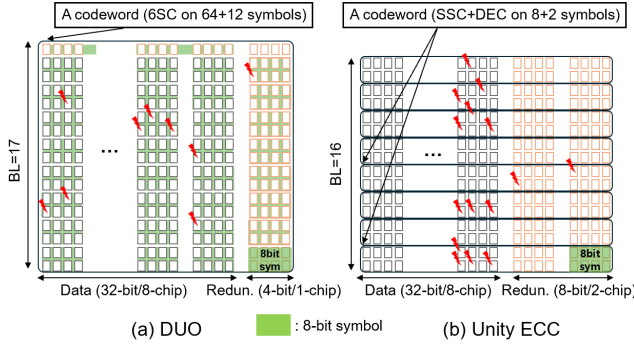


Fig. 3: The single-layer ECC configurations of DDR5

IV. PRIOR WORK

The inefficiency and limited reliability of multi-layer memory protection in commercial DRAMs have inspired numerous academic efforts to redesign ECC architectures. Prior work largely follows two directions: (i) schemes that consolidate protection into a strengthened S-ECC, and (ii) schemes that explicitly coordinate layers and leverage cross-layer information. This section summarizes representative research along these lines.

A. Stronger System ECC

1) *DUO*: DUO [68] bypasses on-die ECC and repurposes its internal redundancy at the system level. By forwarding redundant bits to the host through additional transfer beats, DUO extends S-ECC into a longer symbol-based codeword (e.g., RS(76,64)), providing SDDC protection. A portion of this redundancy is reserved as *on-chip redundancy parity*, which supports burst-erasure correction and verification, enabling recovery even when a complete chip failure coincides with multiple hard defects (Fig. 3a).

2) *Unity ECC*: Unity ECC [22] presents a single-layer S-ECC framework capable of correcting both double-bit errors and single-chip failures. By allowing S-ECC to handle both frequent bit faults and rare device-level errors, Unity ECC

eliminates the need for on-die correction. This simplification reduces latency, power, and area overheads associated with O-ECC while maintaining robust end-to-end protection (Fig. 3b).

3) *Dual-Axis ECC*: Dual-Axis ECC [69] protects both storage and transfer errors via two-orientation decoding: vertical symbols correct storage faults, while horizontal overlays correct common DQS-induced transfer errors by reusing spare syndromes, eliminating L-ECC overhead and reducing retransmissions without extra redundancy.

B. Cross-layer Collaboration

1) *XED*: XED [65] proposes cooperative interaction between S-ECC and O-ECC to achieve SDDC protection. It exposes on-die ECC detection outcomes to the system through a predefined *catch-word* interface, enabling S-ECC to perform erasure-based decoding with improved correction efficiency in configurations with multiple DRAM devices per channel.

2) *HARP*: HARP [70] improves reliability through profiling-based repair under O-ECC. It leverages O-ECC's information (e.g., via decode-bypass and/or correction-event reporting) to profile vulnerable locations and then applies repair actions to mitigate future errors at those locations. Its benefit depends on profiling coverage and available repair resources.

Table I summarizes industry practice and two representative academic designs (DUO and Unity ECC) for single-device-per-channel organizations. Collectively, these academic approaches demonstrate the potential of single-layer ECC to improve system reliability and reduce redundancy. However, they overlook a critical industry requirement—*error concealment within DRAM devices*. To preserve product quality perception, DRAM vendors deliberately mask internal fault behavior, reporting only a limited set of vulnerable regions rather than exposing raw error counts [71]. Although academic proposals enhance transparency and end-to-end reliability, their exposure of device-level errors to the host conflicts with this industrial practice, raising concerns over warranty obligations and vendor accountability.

V. CERBERUS

This section introduces Cerberus, a unified, *cross-layer* ECC framework that delivers high reliability at low overhead with robust end-to-end protection. Unlike conventional multi-layer schemes in which each layer is designed and operated independently, Cerberus *co-designs* the generator and parity-check matrices across layers to ensure interoperability. This interoperability allows a single encoder to produce redundancy shared across layers—each serving a distinct purpose (link protection, error concealment, or ultra-high reliability)—while preventing destructive cross-layer interference (e.g., avoiding miscorrections that amplify errors for the next layer). The following sections present the Cerberus architecture and its operational flow.

A. Architecture

Cerberus targets single-device memory channels in which the channel transfer unit matches the device’s internal access unit (e.g., HBM, LPDDR). In this paper, we focus on an HBM configuration with a 256-bit (32B) access unit and a total redundancy budget of 12.5% (4B per 32B data block). The 32B access unit aligns with current HBM practice, whereas the redundancy budget is lower than the HBM baseline (18.8%). Despite using less redundancy, Cerberus delivers stronger protection, making it suitable for custom HBMs and future HBM generations.

Fig. 4 illustrates the overall architecture. The framework consists of a single shared encoder (1) and three layer-specific decoders (1, 2, 3) that cooperate along the data path. The 32-bit redundancy generated once by the encoder is reused—wholly or partially—by the following decoding layers: (1) the *Link Layer* (LL), which provides early detection of write-path link errors; (2) the *Device Layer* (DL), which performs on-die error correction and concealment; and (3) the *System Layer* (SL), which ensures strong end-to-end, system-level reliability. Each layer interprets the shared 32-bit redundancy according to its role. The LL utilizes 16 bits exclusively for error detection; the DL reuses that same 16-bit portion for bit-level single-error correction (SEC) within the die; and the SL leverages the full 32 bits to perform symbol-based correction and detection. This *Encode-Once, Decode-Many* (EODM) organization with hierarchical redundancy reuse eliminates repeated encoding stages, reduces latency, storage overheads, and preserves seamless protection coverage across all layers without reliability gaps.

B. Cerberus Operations

1) *Write Operation*: On a write, the encoder (1) takes 256-bit user data D and appends 32 bits of redundancy composed of R_1 and R_2 (16 bits each). This redundancy is produced by a generator matrix G_{S-ECC} , which is a product of two sub-matrices, G_1 and G_2 ($G_{S-ECC} = G_1 \cdot G_2$). G_1 first maps 256-bit D to a 272-bit intermediate codeword $(D+R_1)$; G_2 then maps this 272-bit word to a 288-bit final codeword $((D+R_1)+R_2)$. Although the description separates these as two encoding stages for conceptual clarity, practical

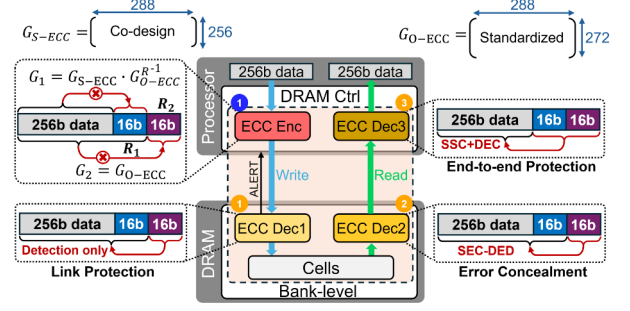


Fig. 4: An overview of Cerberus

implementations can perform both mappings in a single step using the composite matrix G_{S-ECC} .

The host omits conventional write-path L-ECC and transmits the 288-bit codeword $((D+R_1)+R_2)$ directly to DRAM. On the DRAM side, the first decoder (1) replaces L-ECC by verifying the link integrity of $((D+R_1)+R_2)$ using R_2 via the parity-check matrix H_2 (the dual of G_2). Upon a mismatch, the decoder does not attempt correction but instead requests re-transmission (e.g., via a conventional ALERT signal). The 16-bit redundancy provides early detection of transfer errors with a detection capability comparable to CRC16. If verification succeeds, the validated codeword $((D+R_1)+R_2)$ is stored in DRAM with its redundancy preserved for subsequent device- and system-level reuse.

2) *Read Operation*: On a read, the second decoder (2) operates inside the bank group to perform *on-die* error correction and concealment. The device validates $((D+R_1)+R_2)$ using the same parity-check matrix H_2 , but this time, it applies the correction (SEC) when needed. This ensures that single-bit errors are corrected within the device, preventing their propagation to the memory controller.

Importantly, H_2 is designed to satisfy a *bounded-fault* constraint at the 16-bit symbol granularity: any miscorrection that arises from multiple flipped bits is *confined* to the originally faulty 16-bit symbol and cannot corrupt additional symbols. Consequently, device-level actions never increase symbol-level error severity. After local correction, the DRAM bypasses read-path L-ECC generation and instead forwards the corrected 288-bit codeword $((D+R_1)+R_2)$ to the controller.

At the controller, the third decoder (3) provides system-level protection using both redundancy fields (R_1, R_2). Leveraging the full 32-bit redundancy, it delivers SSC+DEC capability: it corrects either a single 16-bit symbol error (e.g., from a device-level miscorrection or critical peripheral-circuit faults) or two bit errors located in distinct symbols (e.g., overlapping storage/transfer errors). For more severe patterns, it still provides safe detection with probability 99.97%. It also extends correction beyond bank-group boundaries and replaces read-side L-ECC. When the decoder detects an uncorrectable error, it issues a single retry. If the retry is still uncorrectable, it reports a DUE. Otherwise, it treats the first event as a transient read-link or peripheral error and forwards the retry result, as in conventional L-ECC retry.

Collectively, Cerberus delivers SSC+DEC correction to the system, SEC correction to DRAM devices, and CRC16-level detection for link protection, meeting the distinct requirements of DRAM reliability and manufacturability. Compared to typical HBM designs, Cerberus offers stronger end-to-end correction at 12.5% overhead (vs. 18.8%) and is not confined to a single bank group. Compared to LPDDR-style protection, it matches the 12.5% overhead but upgrades from bit-level SEC to symbol-aware SSC+DEC at the host, while retaining on-die SEC and link detection for full end-to-end coverage.

C. Cross-Layer ECC Collaboration

This architecture raises a key question: how can the system and device layers collaborate so that redundancy provisioned for the system layer is reusable in the device layer? We address this question in the following subsections by describing our device/link-layer, system-layer, and cross-layer ECC designs.

1) *Device/Link Layer*: We begin by describing the decoding mechanism at the device and link layers. These layers receive 288-bit data $((D+R_1)+R_2)$ and use R_2 to detect and correct errors in D and R_1 , resulting in a parity-check matrix H_2 with dimensions 16×288 .

The device layer targets SEC-DED with a bounded-fault constraint at 16-bit granularity. We choose SEC based on observations from HBM3/4, where strong on-die correction can trigger severe miscorrections and still leave the system exposed to faults outside the bank group. Such miscorrections then force the system layer to provision additional correction strength and redundancy. For example, an O-ECC using 8-bit SSC with 2-symbol redundancy can miscorrect a two-symbol error into a three-symbol corruption, which would require 6-symbol redundancy in S-ECC for full recovery. Moreover, because error sources are not confined to a single bank group, strong intra-bank protection is helpful but offers limited reliability beyond the bank group. Therefore, we design the device layer to handle frequent small-scale bit errors, while the system layer is responsible for rarer but more severe faults.

To achieve SEC-DED with bounded-fault recovery, H_2 must satisfy the following conditions:

- Every column must be non-zero.
- SEC: All columns must be unique.
- DED: The sum of any two distinct columns must not be equal to any other column.
- Bounded fault: The sum of columns within any 16-column region must not match any column belonging to another region.

Additionally, since H_2 is also used in the link layer, it must be able to detect frequent transfer errors. To achieve this, we design H_2 to meet both the device-layer constraints and the CRC8 requirements. Unlike CRC16, CRC8 does not guarantee detection of all burst errors of length 9–16; however, the combined 16-bit redundancy still provides a random-error detection probability of $1 - 2^{-16}$ ($\approx 99.998\%$). Detecting 16-bit burst errors is especially important in DDR systems, where the burst length is typically 16, whereas in memory systems with shorter bursts (e.g., HBM4 with burst-8), this level of

detection is less critical. To realize CRC8, H_2 must satisfy the condition that any eight consecutive columns are linearly independent.

As a result, we use H_2 to protect both the device and link layers, implementing SEC-DED with a bounded-fault property while also meeting CRC8 and providing CRC16-level detection capability.

2) *System Layer*: The system layer is responsible for ensuring end-to-end reliability by detecting and correcting errors that may have propagated through the device and link layers. This layer is designed to provide robust protection against severe, rare errors that escape the correction capabilities of the device and link layers. The system layer achieves this by leveraging the redundancy provided by the device layer (via R_1 and R_2) to perform symbol-based error correction, leading to a parity-check matrix (H_{S-ECC}) with dimensions of 32×288 .

To achieve this, the system layer is designed to correct single 16-bit symbol errors or double-bit errors (SSC+DEC). This dual capability addresses both clustered errors caused by malfunctioning peripheral circuits (e.g., subwordline driver failures) and frequent, random bit errors that may occur simultaneously. The goal of the system layer is to maintain high levels of protection with minimal additional redundancy overhead, thereby ensuring end-to-end reliability without significant performance or storage costs.

To achieve SSC+DEC, H_{S-ECC} must satisfy the following conditions:

- Every column must be non-zero.
- SSC: The sums of all symbol-aligned columns are unique.
- DEC: The sums of any two columns are unique.
- SSC+DEC: All sums from properties 2 and 3 should be unique (apart from double-bit errors in the same symbol, which are considered symbol errors).

For effective collaboration between the device/link layers and the system layer, H_{S-ECC} must satisfy a single condition, which we discuss in the next section.

3) *Cross Layer*: This section describes how H_{S-ECC} can be decomposed into H_1 and H_2 while encoding only once, and presents the single condition under which any 256-bit granularity DRAM can adopt the Cerberus framework. We begin by describing the shared-encoder generator matrix, G_{S-ECC} .

We design G_{S-ECC} to satisfy $G_{S-ECC} = G_1 \cdot G_2$. For such a G_1 to exist, each row of G_{S-ECC} must be expressible as a linear combination of the rows of G_2 . In other words, the row space of G_{S-ECC} must be contained within the row space of G_2 (i.e., $\text{row}(G_{S-ECC}) \subseteq \text{row}(G_2)$). To ensure this condition in the parity-check matrix domain, we utilize the relationship between the generator matrix G and the parity-check matrix H (e.g., $H_2 G_2^T = 0$). In this domain, the condition is equivalent to the requirement that each row of H_2 be expressible as a linear combination of the rows of H_{S-ECC} (i.e., $\text{row}(H_2) \subseteq \text{row}(H_{S-ECC})$). If this simple condition holds, there are no additional constraints on adopting the Cerberus framework, which allows it to support a wide range of vendor-specific S-ECC schemes with high scalability.

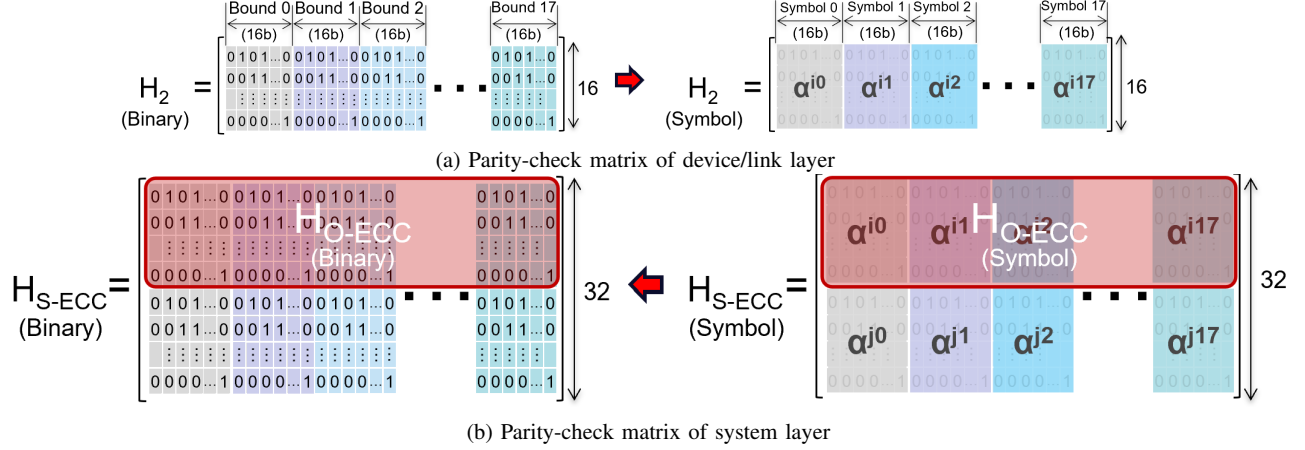


Fig. 5: The parity-check matrices of Cerberus for cross-layer design

D. Code Construction

We derive an H -matrix that satisfies each layer's conditions through a two-step construction. First, we construct H_2 to provide SEC-DED with bounded-faults. We realize SEC-DED by assigning odd-weight columns. Instead of using a prefix region for bounded-faults, we enforce both the bounded fault and CRC8 properties by building the second half (8 columns) of each 16-column bounded region as XOR combinations of the columns in the first half (8 columns). This structure makes it easy to satisfy the CRC8 condition and also helps meet the SSC requirement of H_{S-ECC} . Next, we map each bounded region of the binary H_2 to elements in $GF(2^{16})$ and place this symbolized H_2 (Fig. 5a) directly in the upper part of H_{S-ECC} , thereby satisfying the cross-layer condition ($\text{row}(H_2) \subseteq \text{row}(H_{S-ECC})$).

Second, we construct H_{S-ECC} to satisfy both SSC and DEC. Since H_2 is already placed in the upper part of H_{S-ECC} , we build the lower part using a greedy search. We randomly assign $GF(2^{16})$ elements to each symbol in the lower part, then binarize H_{S-ECC} and check for syndrome overlaps to verify that the SSC and DEC conditions are met. If syndrome overlaps occur, we reconstruct the symbol with the largest number of overlaps and repeat this process until H_{S-ECC} satisfies both SSC and DEC (Fig. 5b).

E. Hardware Implementation

Cerberus largely reuses standard memory ECC primitives (e.g., an encoder and an SEC decoder) and updates only the G and H matrices (Fig. 6). On the write path, the controller encoder (1) computes R_1 and R_2 in a single pass using $G_{S-ECC} = G_1 \cdot G_2$. It implements this multiplication with an XOR network (e.g., 8-level XOR trees). In DRAM, the first decoder (1) verifies writes by regenerating R_2 using a subset of the same XOR network.

On the read path, the second decoder in DRAM (2) generates a syndrome with an XOR-tree network and corrects single-bit errors based on the syndrome, which adds modest logic depth (≈ 4) to the combinational logic. It then forwards

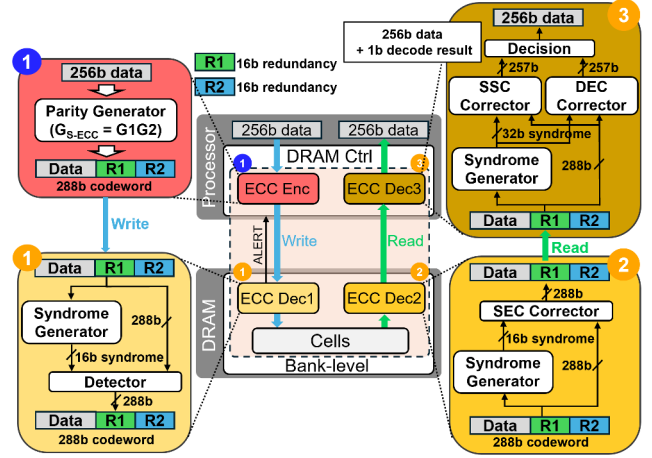


Fig. 6: The hardware implementation of Cerberus

the corrected 288-bit codeword (with redundancy retained) for end-to-end decoding. The third decoder in the controller (3) applies H_{S-ECC} to the received 288-bit codeword to generate a 32-bit syndrome and runs SSC and DEC correctors in parallel [22]. The SSC corrector uses Chien search with a modified Berlekamp–Massey procedure [72], and the DEC corrector uses a block-pair solver [73].

Overall, the encoder and the first two decoders are on par with existing implementations. The main added complexity is the third decoder for SSC+DEC. However, it performs error detection within a single cycle to avoid latency increases on error-free accesses, and it completes correction within a single cycle by running SSC and DEC correction in parallel. Because errors are rare, we include only the detection latency in the performance evaluation (Section VI-B) and report the area overhead of advanced decoding (Section VI-C).

VI. EVALUATION

We evaluate the reliability and performance impact of Cerberus and compare it with single-layer ECC and state-of-the-art multi-layered DRAM ECC configurations.

TABLE II: A comparison of reliability against single-location error scenarios

			Single-layer		Multi-layer				Cross-layer						
Redundancy (%)			12.5% (32b)	18.8% (48b)	12.5% (32b)		18.8% (48b)		12.5% (32b)	15.6% (40b)					
Error Location	Error Scenario	Decoding Result	Unity ECC	DUO	LPDDR6/ SEC-DED	LPDDR6/ CRC	HBM4/ SEC-DED	HBM4/ CRC	Cerberus (32b)	Cerberus (40b)					
In bank	SE (%)	CE	100.000												
	16E (%)	CE	100.000		0.048	0.024	100.000								
		DUE	0.000		99.563	99.976	0.000								
		SDC	0.000		0.389	0.000	0.000								
	32E (%)	CE	0.003	1.558	0.003	0.000	0.003	0.003	0.003	0.003					
		DUE	99.972	98.407	99.585	99.999	99.997	99.997	99.971	99.997					
		SDC	0.025	0.035	0.412	0.001	2×10^{-5}	1×10^{-6}	0.026	3×10^{-4}					
	SE+SE (%)	CE	100.000		10.515	0.000	4.988	4.940	100.000						
Write Link	DUE	0.000		89.485	100.000	94.980	95.060	0.000							
	SDC	0.000		0.000	0.000	0.032	0.000	0.000							
	SE (%)	CE	100.000												
	DQE (%)	CE	100.000												
	DQSE (%)	CE	0.000	0.000	99.998	99.998	49.994	49.991	99.999						
		DUE	99.971	99.945	0.002	0.002	50.005	50.008	0.001						
		SDC	0.029	0.055	0.000	0.000	0.001	0.001	0.000						
	Out bank (Read peri/link)	SE (%)	CE	100.000		100.000	0.000	100.000	0.000	100.000					
DUE			0.000		0.000	100.000	100.000	100.000	0.000						
CE			100.000												
DE (%)		DUE	100.000		0.000				100.000						
		CE	0.000		100.000				0.000						
		DUE	100.000												
DQE (%)		CE	100.000												
DQSE (%)		DUE	0.000	0.000	99.998	99.998	49.994	49.991	99.972	100.000*					
	DUE	99.971	99.945	0.002	0.002	50.005	50.008	0.000	0.000						
	SDC	0.029	0.055	0.000	0.000	0.001	0.001	0.028	1×10^{-4}						

* Rounded to 100.000% for display; actual value is slightly lower (e.g., 99.9998%).

A. Error Coverage

We evaluate the reliability of Cerberus using Monte Carlo error-injection experiments. We inject random errors under various error scenarios, and quantify correction and detection through ECC decoding. Based on system-level analyses of DRAM faults [15]–[17], [58], [74], we derive location-specific error patterns, detailed below.

We consider three error locations: (i) *In bank*, (ii) *Write link*, and (iii) *Out bank*. First, *In bank* covers faults internal to a DRAM bank. We consider the following error scenarios: Single Error (SE; caused by cell or BLSA), 16-bit Error (16E; CSL or SWL), 32-bit Error (32E; SWD) and a combination of two Single Errors (SE+SE). Second, *Write link* denotes the transmit path during writes, including transmission-induced faults. We select error scenarios: SE, Data Pin Error (DQE), and Data Strobe Error (DQSE). Third, *Out bank* spans the read path beyond the bank—device periphery and I/O—and we select error scenarios: SE, Double Error (DE; peripheral errors (e.g., TSV)) and DQE, DQSE (read-link errors).

For each error scenario, we inject errors at the specified DRAM location and flip the designated number of bits, each independently with a 50% probability. We evaluate both single-location and multi-location cases. Each ECC scheme classifies outcomes as correctable (CE), detectable but uncorrectable (DUE), or undetectable (SDC), and we aggregate results over 10 million iterations. For link errors, we count DUEs as CEs due to retransmission and retry.

We compare Cerberus against a range of layered ECC configurations. For single-layer schemes, we use Unity ECC [22] and DUO [68], both deployed on single-device memory. For multi-layer schemes, we evaluate state-of-the-art DRAM configurations, LPDDR6 [25] and HBM4 [26], which employ S-ECC as either SEC-DED or CRC [37], [52]. Finally, we

evaluate the cross-layer framework Cerberus with both 12.5% (32b) and 15.6% (40b) redundancy.

1) *Single-Location*: Table II summarizes error coverage and redundancy for each ECC configuration under single-location scenarios. For the *In bank*, all configurations correct SE and 16E at 100% except for 16E in LPDDR6. Because LPDDR6 partitions the 12.5% redundancy per layer, it lacks sufficient budget to correct 16E. Moreover, LPDDR6/SEC-DED shows a high SDC rate for 16E and 32E due to O-ECC miscorrections. By contrast, Cerberus guarantees 100% correction of 16E while using the same 12.5% total redundancy budget across layers. For other scenarios, Cerberus provides strong detection capability for 32E and, with increased redundancy, can further enhance robustness.

For SE+SE, none of the multi-layer configurations provides guaranteed correction. In LPDDR6, the SEC-DED O-ECC detects the event but forwards the uncorrected data to the controller without a hint, while the downstream S-ECC (SEC-DED or CRC) lacks sufficient correction capability (except in cases where errors occur in on-die parity bits that are not transferred). Similarly, in HBM4, the SSC O-ECC corrects only when both errors fall within the same symbol; otherwise the residual pattern exceeds the S-ECC capability. This behavior aligns with prior observations on multi-layer protection [50], although the SDC rates differ because [50] considers SEC O-ECC (without double-error detection). Unity ECC and DUO correct SE+SE by concentrating redundancy in a strengthened single layer that supports double-bit correction. Cerberus also corrects SE+SE via its SSC+DEC while preserving on-die error concealment for the common case of single-bit errors.

For the *Write link*, Unity ECC and DUO can correct SE and DQE with strong S-ECC, but wider transfer errors (DQSE) are only *detected*, not corrected. Because they are single-layer

TABLE III: A comparison of reliability against multi-location error scenarios

			Single-layer		Multi-layer				Cross-layer	
Redundancy (%)			32b (12.5%)	48b (18.8%)	32b (12.5%)		48b (18.8%)		32b (12.5%)	40b (15.6%)
Error Location	Error Scenario	Decoding Result	Unity ECC	DUO	LPDDR6/ SEC-DED	LPDDR6/ CRC	HBM4/ SEC-DED	HBM4/ CRC	Cerberus (32b)	Cerberus (40b)
In bank+ Out bank	SE+ SE (%)	CE DUE	100.000 0.000		100.000 0.000	0.000 100.000	100.000 0.000	0.000 100.000	100.000 0.000	
	SE+ DE (%)	CE DUE SDC	0.947 99.019 0.034	100.000 0.000 0.000	0.000 100.000 0.000				100.000 0.000 0.000	
In bank+ Write Link	16E+ DQE (%)	CE DUE SDC	5.545 94.431 0.024	100.000 0.000 0.000	0.047 99.565 0.388	0.025 99.975 0.000	100.000 0.000 0.000			
	32E+ DQSE (%)	CE DUE SDC	0.000 99.972 0.028	0.000 99.950 0.050	0.002 99.586 0.412	0.000 99.999 0.001	0.001 99.999 2×10^{-5}	0.001 99.999 8×10^{-7}	0.003 99.972 0.025	0.003 99.997 3×10^{-4}
In bank+ Out bank+ Write Link	SE+ SE+ SE (%)	CE DUE SDC	1.293 98.672 0.035	100.000 0.000 0.000	100.000 0.000 0.000	0.000 100.000 0.000	100.000 0.000 0.000	0.000 100.000 0.000	100.000 0.000 0.000	
	SE+ DQE+ DQSE (%)	CE DUE SDC	0.000 97.971 0.028	0.000 99.950 0.050	99.998 0.002 3×10^{-5}	99.998 0.002 0.000	50.010 49.988 0.001	49.982 50.017 0.001	99.998 0.002 0.000	99.999 0.001 0.000

schemes, they cannot perform early detection, so corrupted data may be written back uncorrected, allowing subsequent faults to accumulate and increase the risk of severe reliability issues. In contrast, the multi-layer baselines employ L-ECC and thus offer high detection for SE/DQE/DQSE, enabling correction via retransmission. However, when the L-ECC is provisioned with limited redundancy, as in HBM4, DQSE detection drops to roughly half of cases. Cerberus, on the other hand, guarantees 100% detection (and thus correction) for SE and DQE, and for wider DQSE, the first decoder already provides high detection; any remaining cases are caught by the stronger third decoder, yielding a robust end-to-end design.

For the *Out bank*, Unity ECC and DUO can correct SE, DE, and DQE using a strong S-ECC. However, they only *detect* DQSE because they do not guarantee a retry after detection. In contrast, the multi-layer configurations do not guarantee correction for peripheral DE. Moreover, when they apply S-ECC with CRC (LPDDR6-CRC and HBM4-CRC), they fail to correct even SE and instead provide detection only. This limitation stems from redundancy partitioning across layers, which prevents redundancy reuse. For example, HBM4 can allocate 32b to a strong O-ECC (SSC) and leave only 16b for S-ECC. This allocation forces weaker codes (SEC-DED or CRC) and leaves the system vulnerable to out-of-bank errors. In contrast, Cerberus guarantees 100% correction for SE, DE, and DQE, and achieves near-complete correction for DQSE with retries.

2) *Multi-Location*: Table III summarizes the error coverage and redundancy of each ECC configuration under multi-location scenarios. For combined *In bank* and *Out bank* errors, Unity ECC and DUO guarantee 100% correction for SE+SE. However, for SE+DE, DUO still corrects while Unity ECC fails. Although Unity ECC corrects SE and DE individually in the single-location case, the overlap across two locations exposes the limitation of relying solely on S-ECC. The multi-layer approaches show the same behavior as in the single-location *Out bank* case (e.g., failing to correct peripheral DE), because although O-ECC corrects *In bank* SE, *Out bank* errors still remain due to the weaker code used for S-ECC.

In contrast, Cerberus guarantees correction for both SE+SE and SE+DE across locations even with a small redundancy budget, as O-ECC corrects the *In bank* SE and S-ECC covers all *Out bank* errors. For combined *In bank* and *Write link*, configurations with 12.5% redundancy (Unity ECC/LPDDR6) do not guarantee correction for 16E+DQE. However, Cerberus can correct it even with the same redundancy. For 32E+DQSE, Cerberus likewise provides significantly higher detection than other schemes with the same redundancy. Finally, in the high-risk scenario where errors occur simultaneously at *all* locations (*In bank*, *Out bank*, and *Write link*), single-layer configurations reveal the limitation of relying only on S-ECC, since all overlapping errors are exposed to the system layer. DUO can correct SE+SE+SE with its stronger scheme, but it fails once larger errors are involved, and Unity ECC also cannot guarantee correction across these cases. Multi-layer configurations improve locality by letting each layer handle its corresponding errors (e.g., O-ECC handles *In bank* errors and L-ECC handles link errors), but because the limited redundancy budget is divided across layers, they still fail to provide correction in all cases. In contrast, Cerberus retains the role of each layer while reusing redundancy across layers, enabling nearly 100% correction across all cases with a small redundancy budget and thus offering robust end-to-end reliability.

Rather than limiting our evaluation to in-DRAM faults, we assess reliability using error scenarios that span the entire memory system, and show that Cerberus maintains strong reliability under both single- and multi-location errors. In addition, Cerberus is a scalable framework that can accommodate higher redundancy. Although this increases overhead, it improves detection capability and yields a more robust system.

B. Performance & Energy Consumption

1) *GPU Performance*: We evaluate the performance impact of Cerberus (32b) and Cerberus (40b) on GPUs using the cycle-level simulator Accel-Sim [75], and compare it against HBM4 [26], Unity ECC [22], and DUO [68]. Our system model is based on an NVIDIA V100 GPU configured with 32 HBM channels, with detailed parameters listed in Table IV.

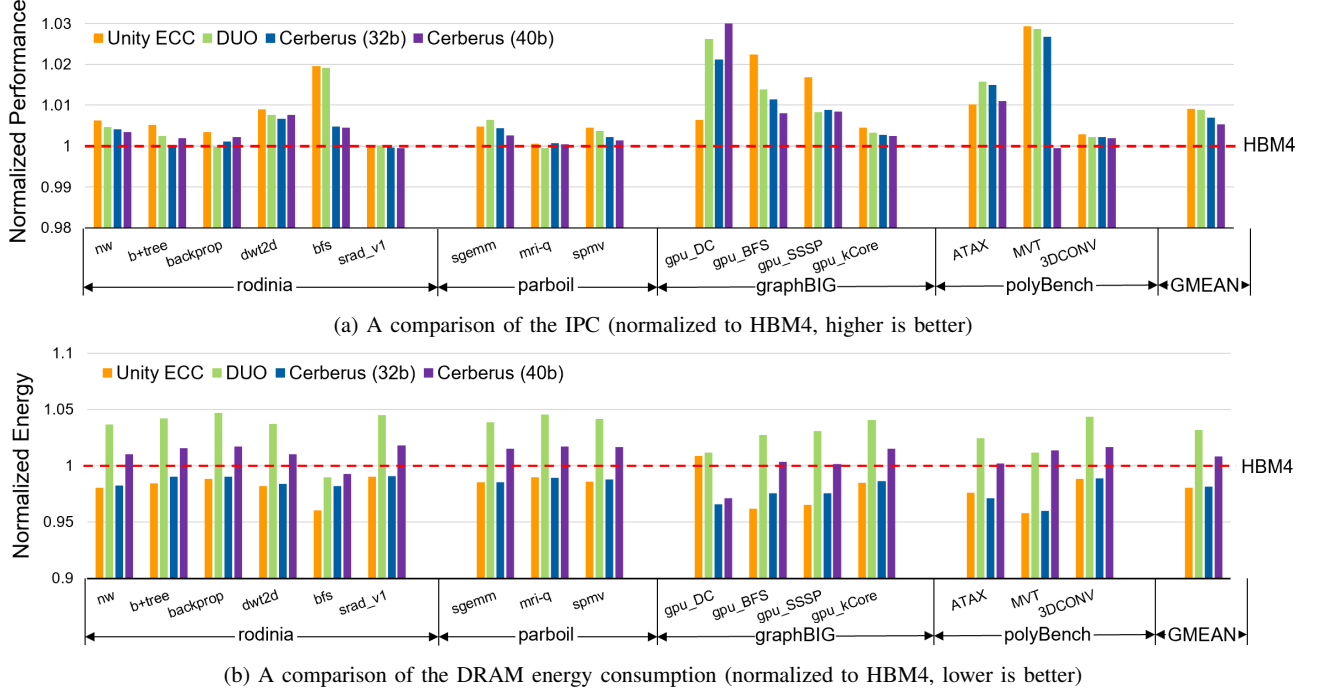


Fig. 7: Comparison of GPU performance and DRAM energy for Cerberus across the evaluated benchmarks

TABLE IV: The simulation configuration

Components	Configuration
# of SMs	80
SM	1132 MHz, 4 warp schedulers/SM, up to 32 blocks/SM, up to 48 warps/SM
L1 cache	Up to 128KiB, 4 banks, 128B line, 256-way, 384 MSHR entries
L2 cache	4MiB, 128B lines, 16-way, 192 MSHR entries
Memory controller	256B channel interleaving, 64-entry scheduling queue, FR-FCFS scheduling
Memory	HBM4, 32 channels, 6.4Gbps tRCD = 30-cycle, tRRD = 4-cycle, CL = 24-cycle, tCCDS = 2-cycle, tCCDL = 4-cycle, WL = 14-cycle

To capture a wide range of application behaviors, we use 16 workloads drawn from four benchmark suites: Rodinia [76], Parboil [77], GraphBIG [78], and PolyBench [79].

The ECC decoder affects the time from a read command to the output of the first corresponding data beat (tCL), while the encoder impacts the write latency (tWL) [80]. We evaluate the performance by adjusting these two timing parameters for each ECC configuration. For HBM4, we estimate the encoder and decoder latency overheads of 16-bit SSC-based O-ECC as 2ns and 5ns, respectively [80]. In contrast, the latency overheads of both S-ECC (CRC) and L-ECC (parity) are less than 1ns for both encoding and decoding. For Unity ECC and DUO, we derive their timing parameters by first removing the O-ECC latency overhead from the HBM4 baseline and then adding each scheme’s synthesized latency. Logic synthesis in a UMC 28nm process yields latency overheads of 1.46ns for Unity ECC and 1.92ns for DUO. For both Cerberus (32b) and Cerberus (40b), we remove the O-ECC encoder latency and

reduce the decoder latency from 5ns to 2ns under the modified O-ECC scheme [80], since the two configurations have nearly identical O-ECC latency. We also model S-ECC latency from logic synthesis, adding 0.85ns for Cerberus (32b) and 0.89ns for Cerberus (40b). Finally, we convert all latency values into clock cycles at 1.6 GHz (6.4 Gbps after QDR in HBM4).

Fig. 7a reports the instructions per cycle (IPC) for HBM4, Unity ECC, DUO, Cerberus (32b) and Cerberus (40b), normalized to HBM4. Across benchmarks, Cerberus (32b) improves IPC by 0.2%, 0.2%, 1.1%, and 1.4% (0.7% geomean), while Cerberus (40b) improves IPC by 0.3%, 0.1%, 1.2%, and 0.4% (0.5% geomean). Unity ECC and DUO achieve similar IPC gains, but they provide lower reliability than Cerberus. In contrast, Cerberus delivers higher reliability than HBM4 while using less redundancy, yet still improves IPC. It also attains performance comparable to Unity ECC (0.9% geomean). These benefits stem from the EODM organization of Cerberus, which eliminates repeated encoding stages through efficient reuse of redundancy across layers.

2) *DRAM Energy Consumption*: Cerberus uses different storage and transfer bit widths than HBM4. To estimate the resulting DRAM power/energy, we use HBM2E operating currents from a datasheet [81].

We assume the precharge standby current (IDD2N) is independent of bit width, while the incremental activation current (IDD0–IDD2N) and active-standby current (IDD3N–IDD2N) scale with the stored bit count (e.g., (256+32+16) bits in HBM4 vs. (256+32) bits in Cerberus (32b)). For read/write activity, we partition the incremen-

TABLE V: Estimated DRAM operating currents (per pseudo-channel)

DRAM current	Unity ECC	DUO	HBM4	Cerberus (32b)	Cerberus (40b)
IDD0 (mA) (ACT-PRE)	47.42 (99.7%)	47.56 (100%)	47.56 (100%)	47.42 (99.7%)	47.49 (99.9%)
IDD2N (mA) (Precharge standby)			44.88 (100%)		
IDD3N (mA) (Active standby)	47.69 (99.7%)	47.84 (100%)	47.84 (100%)	47.69 (99.7%)	47.77 (99.8%)
IDD4R (mA) (Read)	521.28 (99.2%)	547.75 (104.2%)	525.72 (100%)	521.28 (99.2%)	534.52 (101.7%)
IDD4W (mA) (Write)	365.27 (99.2%)	383.07 (104%)	368.25 (100%)	365.29 (99.2%)	374.17 (101.6%)

tal currents (IDD4R–IDD2N and IDD4W–IDD2N) between bank-group-internal transfer (cells→O-ECC) and bank-group-external transfer (O-ECC→processor) and apply a 61%:39% split from prior HBM2 analysis [8]. We then scale each component by the corresponding transfer width: HBM4 transfers (256+32+16) bits within a bank group and (256+16) bits outside, whereas Cerberus (32b) transfers (256+32) bits within a bank group and (256+32) bits outside. Finally, we compute overall DRAM energy consumption using the Micron DDR4 power calculator [82] with these current values. Table V summarizes the DRAM operating currents used in our evaluation.

Fig. 7b shows the results. Cerberus (32b) reduces energy by 1.84% on average compared to HBM4. This is primarily because intra-die transfers from cells to bank peripherals often consume more energy than off-chip transfer [8], [83], and Cerberus reduces these bank-group–internal transfers. With higher redundancy, Cerberus (40b) consumes 0.86% more energy than HBM4 on average to provide stronger protection.

C. Hardware Overheads

To estimate the hardware costs, we implement SystemVerilog models for the encoder and decoders. We synthesize these models with Synopsys Design Compiler using a UMC 28nm standard library. We then normalize the resulting area to NAND2 equivalents (the number of NAND2 gates that occupy the same area) to present process-independent results.

Table VI summarizes the area cost of Cerberus. Decoder 1 (L-ECC) and Decoder 2 (O-ECC) reside inside the DRAM device. Together, they require only 14,693 NAND2 equivalents in Cerberus (32b) and 15,662 NAND2 equivalents in Cerberus (40b), which correspond to 0.0074 mm² and 0.0079 mm², respectively. This overhead is negligible compared to an HBM stack footprint (e.g., 121 mm² for HBM3 [84]). Decoder 3 (S-ECC) dominates the overall overhead, mainly due to the DEC corrector. Even so, the processor-side overheads (encoder+S-ECC) total 127,583 NAND2 equivalents for Cerberus (32b) and 167,954 NAND2 equivalents for Cerberus (40b). Relative to modern GPUs with billions of transistors, this is a tiny fraction of the transistor count (e.g., 2.5×10^{-6} to 3.2×10^{-6} of a 208B-transistor Blackwell [85]).

TABLE VI: Area overheads (in NAND2 equivalents)

	Cerberus (32b)	Cerberus (40b)
Encoder	1632.79 μm^2 (3240)	2015.49 μm^2 (3999)
Decoder 1 (L-ECC)	1205.40 μm^2 (2392)	1398.43 μm^2 (2775)
Decoder 2 (O-ECC)	6199.87 μm^2 (12301)	6495.15 μm^2 (12887)
Decoder 3 (S-ECC)	62669.04 μm^2 (124343)	82633.15 μm^2 (163955)

VII. CONCLUSION

This paper presents Cerberus, a cross-layer ECC co-design that addresses key challenges of multi-layer ECC: inefficient use of redundancy, overlapping protection coverage, and destructive cross-layer interference due to miscorrections. Implemented on HBM4 with a cross-layer ECC design, Cerberus reduces redundancy by 33.3% while still providing higher reliability through efficient redundancy reuse. Moreover, its Encode-Once, Decode-Many (EODM) architecture eliminates unnecessary encoding stages, improving performance and delivering seamless coverage without protection gaps. Overall, Cerberus provides a promising framework for achieving high reliability in future HBM- and LPDDR-based systems.

ACKNOWLEDGMENT

This work was supported in part by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2026-25519569, Development of an Operating Interface and Platform for LPDDR6-PIM, 20%; No. RS-2026-25520910, Development of LPDDR6 Interface, Low-power and High-speed Memory Network, 20%; No. RS-2019-II190421, AI Graduate School Support Program, Sungkyunkwan University, 20%; No. RS-2023-00228970, Development of Flexible SW-HW Conjunctive Solution for On-edge Self-supervised Learning, 20%; No. RS-2025-10692981, AI Semiconductor Innovation Research Center, 20%).

REFERENCES

- [1] A. Spessot and H. Oh, “1T-1C Dynamic Random Access Memory Status, Challenges, and Prospects,” *IEEE Transactions on Electron Devices*, vol. 67, no. 4, 2020.
- [2] S.-L. Gong, J. Kim, and M. Erez, “DRAM Scaling Error Evaluation Model Using Various Retention Time,” in *Proceedings of the Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*, 2017.
- [3] O. Mutlu, *Main Memory Scaling: Challenges and Solution Directions*. Springer New York, 2015.
- [4] H. Hassan, M. Patel, J. S. Kim, A. G. Yaglikci, N. Vijaykumar, N. M. Ghiasi, S. Ghose, and O. Mutlu, “CROW: A Low-Cost Substrate for Improving DRAM Performance, Energy Efficiency, and Reliability,” in *Proceedings of the 46th international symposium on computer architecture*, 2019.
- [5] H. Ha, *Understanding and Improving the Energy Efficiency of DRAM*. Stanford University, 2018.
- [6] H. Park, S.-M. Yu, and J. Song, “An 11 Gb/s 0.376 pJ/Bit Capacitor-Less Dicode Transceiver With Pattern-Dependent Equalizations TIA Termination for Parallel DRAM Interfaces,” *IEEE Access*, 2024.
- [7] Y. Jung, S. Lee, H. Kim, and S. Cho, “A Supply-Noise-Induced Jitter-Cancelling Clock Distribution Network for LPDDR5 Mobile DRAM featuring a 2nd-order Adaptive Filter,” in *Proceedings of the International Solid State Circuits Conference (ISSCC)*, vol. 65, 2022.

- [8] M. O'Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, "Fine-Grained DRAM: Energy-Efficient DRAM for Extreme Bandwidth Systems," in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, 2017.
- [9] K. S. Yim, C. Pham, M. Saleheen, Z. Kalbarczyk, and R. Iyer, "Hauverk: Lightweight Silent Data Corruption Error Detector for GPGPU," in *Proceedings of the International Symposium on Parallel and Distributed Processing (IPDPS)*, 2011.
- [10] D. Fiala, F. Mueller, C. Engelmann, R. Riesen, K. Ferreira, and R. Brightwell, "Detection and Correction of Silent Data Corruption for Large-Scale High-Performance Computing," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC)*, 2012.
- [11] R. Yelleswarapu and A. K. Somani, "Addressing multiple bit/symbol errors in DRAM subsystem," *PeerJ Computer Science*, vol. 7, 2021.
- [12] M. B. Sullivan, M. T. I. Ziad, A. Jaleel, and S. W. Keckler, "Implicit Memory Tagging: No-Overhead Memory Safety Using Alias-Free Tagged ECC," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2023.
- [13] S. Li, D. H. Yoon, K. Chen, J. Zhao, J. H. Ahn, J. B. Brockman, Y. Xie, and N. P. Jouppi, "MAGE: Adaptive Granularity and ECC for Resilient and Power Efficient Memory Systems," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, 2012.
- [14] B. Salami, O. S. Unsal, and A. C. Kestelman, "Evaluating Built-In ECC of FPGA On-Chip Memories for the Mitigation of Undervolting Faults," in *Proceedings of the 27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, 2019.
- [15] M. V. Beigi, Y. Cao, S. Gurumurthi, C. Recchia, A. Walton, and V. Sridharan, "A Systematic Study of DDR4 DRAM Faults in the Field," in *Proceedings of the International Symposium on High Performance Computer Architecture (HPCA)*, 2023.
- [16] H. Chung, E. Oh, S. Baek, H. Yoon, J. Yoo, S. Lee, Y. Lee, A. Bramhanand, B. Dodds, Y. Zhou, and N. S. Kim, "DRAM Fault Classification through Large-Scale Field Monitoring for Robust Memory RAS Management," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2025.
- [17] J. Jung and M. Erez, "Predicting Future-System Reliability with a Component-Level DRAM Fault Model," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023.
- [18] A. Bacchini, M. Rovatti, G. Furano, and M. Ottavi, "Characterization of Data Retention Faults in DRAM Devices," in *Proceedings of the International Symposium on Defect and Fault Tolerance in VLSI Systems (DFT)*, 2014.
- [19] Y. Moon, S. Shin, S. Jang, D. Won, and S. Kang, "A Novel Prediction-Based Two-Tiered ECC for Mitigating SWD Errors in HBM," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2024.
- [20] M. Patel, J. S. Kim, H. Hassan, and O. Mutlu, "Understanding and Modeling On-Die Error Correction in Modern DRAM: An Experimental Study using Real Devices," in *Proceedings of the International Conference on Dependable Systems and Networks (DSN)*, 2019.
- [21] M. Patel, J. S. Kim, T. Shahroodi, H. Hassan, and O. Mutlu, "Bit-Exact ECC Recovery (BEER): Determining DRAM On-Die ECC Functions by Exploiting DRAM Data Retention Characteristics," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2020.
- [22] D. Kim, J. Lee, W. Jung, M. B. Sullivan, and J. Kim, "Unity ECC: Unified Memory Protection Against Bit and Chip Errors," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC)*, 2023.
- [23] R. Xie, A. U. Haq, Y. Fang, L. Ma, S. Sen, S. Venkataramani, L. Liu, and T. Zhang, "Breaking the HBM Bit Cost Barrier: Domain-Specific ECC for AI Inference Infrastructure," *IEEE Computer Architecture Letters*, 2025.
- [24] JEDEC standard, "Double Data Rate (DDR) 5," in *JESD79-5C.01*, 2024.
- [25] —, "LPDDR6 standard," in *JESD209-6*, 2024.
- [26] —, "High Bandwidth Memory (HBM4) DRAM," in *JESD270-4*, 2025.
- [27] D. H. Yoon and M. Erez, "Virtualized and Flexible ECC for Main Memory," in *Proceedings of the fifteenth International Conference on Architectural support for programming languages and operating systems*, 2010.
- [28] K. Criss, K. Bains, R. Agarwal, T. Bennett, T. Grunzke, J. K. Kim, H. Chung, and M. Jung, "Improving Memory Reliability by Bounding DRAM Faults: DDR5 improved reliability features," in *Proceedings of the International Symposium on Memory Systems (MEMSYS)*, 2020.
- [29] L. Cojocar, K. Razavi, C. Giuffrida, and H. Bos, "Exploiting Correcting Codes: On the Effectiveness of ECC Memory Against Rowhammer Attacks," in *Proceedings of the IEEE Symposium on Security and Privacy (SP)*, 2019.
- [30] G. I. Davida and S. M. Reddy, "Forward-Error Correction with Decision Feedback," *Information and Control*, vol. 21, no. 2, 1972.
- [31] T. J. Holman, "Error correction and detection for faults on time multiplexed data lines," Apr. 2001, U.S. Patent 6,219,817.
- [32] J. Kim, M. Sullivan, and M. Erez, "Bamboo ECC: Strong, Safe, and Flexible Codes for Reliable Computer Memory," in *Proceedings of the International Symposium on High Performance Computer Architecture (HPCA)*, 2015.
- [33] A. Singh, S. Chakravarty, G. Papadimitriou, and D. Gizopoulos, "Silent Data Errors: Sources, Detection, and Modeling," in *Proceedings of the VLSI Test Symposium (VTS)*, 2023.
- [34] H. D. Dixit, L. Boyle, G. Vunnam, S. Pendharkar, M. Beadon, and S. Sankar, "Detecting silent data corruptions in the wild," *arXiv preprint arXiv:2203.08989*, 2022.
- [35] D. Agiakatsikas, G. Papadimitriou, V. Karakostas, D. Gizopoulos, M. Psarakis, C. Bélanger-Champagne, and E. Blackmore, "Impact of Voltage Scaling on Soft Errors Susceptibility of Multicore Server CPUs," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023.
- [36] R. W. Hamming, "Error Detecting and Error Correcting Codes," *Bell System Technical Journal*, vol. 29, no. 2, 1950.
- [37] M. Y. Hsiao, "A Class of Optimal Minimum Odd-weight-column SECDED Codes," *IBM Journal of Research and Development*, vol. 14, no. 4, 1970.
- [38] R. C. Bose and D. K. Ray-Chaudhuri, "On A Class of Error Correcting Binary Group Codes," *Information and Control*, vol. 3, no. 1, 1960.
- [39] I. S. Reed and G. Solomon, "Polynomial Codes over Certain Finite Fields," *Journal of the Society for Industrial and Applied Mathematics*, vol. 8, no. 2, 1960.
- [40] A. N. Udipi, N. Muralimanohar, R. Balsubramanian, A. Davis, and N. P. Jouppi, "LOT-ECC: Localized and Tiered Reliability Mechanisms for Commodity Memory Systems," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2012.
- [41] S. Jeong, S. Kang, and J.-S. Yang, "PAIR: Pin-aligned In-DRAM ECC architecture using expandability of Reed-Solomon code," in *Proceedings of the Design Automation Conference (DAC)*, 2020.
- [42] S. Sonawane and V. S. Baste, "Implementation of RS-CC Encoder and Decoder using MATLAB," *International Journal of Science Technology and Engineering*, vol. 5, 2019.
- [43] C. Chen, "Error-correcting codes for semiconductor memories," in *Proceedings of the 11th annual international symposium on Computer architecture*, 1984.
- [44] M. Hsiao, W. C. Carter, J. W. Thomas, and W. R. Stringfellow, "Reliability, Availability, and Serviceability of IBM Computer Systems: A Quarter Century of Progress," *IBM Journal of Research and Development*, vol. 25, no. 5, 1981.
- [45] Synopsys, "Error Correction Code (ECC) in DDR Memories," <https://www.synopsys.com/articles/ecc-memory-error-correction.html>, 2020.
- [46] Advanced Micro Devices, Inc., *BIOS and Kernel Developer's Guide (BKDG) for AMD Family 15h Models 00h-0Fh Processors*, 2013.
- [47] C. Li, Y. Zhang, J. Wang, H. Chen, X. Liu, T. Huang, L. Peng, S. Zhou, L. Wang, and S. Ge, "From Correctable Memory Errors to Uncorrectable Memory Errors: What Error Bits Tell," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC)*, 2022.
- [48] X. Du, C. Li, S. Zhou, X. Liu, X. Xu, T. Wang, and S. Ge, "Fault-Aware Prediction-Guided Page Offlining for Uncorrectable Memory Error Prevention," in *Proceedings of the International Conference on Computer Design (ICCD)*, 2021.
- [49] U. Kang, H.-s. Yu, C. Park, H. Zheng, J. Halbert, K. Bains, S. Jang, and J. S. Choi, "Co-Architecting Controllers and DRAM to Enhance DRAM Process Scaling," in *The Memory Forum*, vol. 14, 2014.
- [50] I. Alam and P. Gupta, "COMET: On-die and In-controller Collaborative Memory ECC Technique for Safer and Stronger Correction of DRAM Errors," in *Proceedings of the International Conference on Dependable Systems and Networks (DSN)*, 2022.
- [51] K. C. Chun, Y. K. Kim, Y. Ryu, J. Park, C. S. Oh, Y. Y. Byun, S. Y. Kim, D. H. Shin, J. G. Lee, B.-K. Ho, M.-S. Park, S.-J. Cho, S. Woo, B. M.

- Moon, B. Kil, S. Ahn, J. H. Lee, S. Y. Kim, S.-K. Choi, J.-S. Jeong, S.-G. Ahn, J. Kim, J. J. Kong, K. Sohn, N. S. Kim, and J.-B. Lee, "A 16-GB 640-GB/s HBM2E DRAM with a Data-Bus Window Extension Technique and a Synergetic On-Die ECC Scheme," *IEEE Journal of Solid-State Circuits*, vol. 56, no. 1, 2020.
- [52] Y. Ryu, S.-G. Ahn, J. H. Lee, J. Park, Y. K. Kim, H. Kim, Y. G. Song, H.-W. Cho, S. Cho, S. H. Song, H. Lee, U. Shin, J. Ahn, J.-M. Ryu, S. Lee, K.-H. Lim, J. Lee, J. H. Park, J.-S. Jeong, S. Joo, D. Cho, S. Y. Kim, M. Lee, H. Kim, M. Kim, J.-S. Kim, J. Kim, H. G. Kang, M.-K. Lee, S.-R. Kim, Y.-C. Kwon, Y. Y. Byun, K. Lee, S. Park, J. Youn, M.-O. Kim, K. Sohn, S.-J. Hwang, and J. Lee, "A 16 GB 1024 GB/s HBM3 DRAM With Source-Synchronized Bus Design and On-Die Error Control Scheme for Enhanced RAS Features," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 4, 2023.
- [53] T.-Y. Oh, H. Chung, J.-Y. Park, K.-W. Lee, S. Oh, S.-Y. Doo, H.-J. Kim, C. Lee, H.-R. Kim, J.-H. Lee, J.-I. Lee, K.-S. Ha, Y. Choi, Y.-C. Cho, Y.-C. Bae, T. Jang, C. Park, K. Park, S. Jang, and J. S. Choi, "A 3.2 Gbps/pin 8 Gbit 1.0 V LPDDR4 SDRAM With Integrated ECC Engine for Sub-1 V DRAM Core Operation," *IEEE Journal of Solid-State Circuits*, vol. 50, no. 1, 2014.
- [54] M.-J. Park, J. Lee, K. Cho, J. Park, J. Moon, S.-H. Lee, T.-K. Kim, S. Oh, S. Choi, Y. Choi *et al.*, "A 192-Gb 12-High 896-GB/s HBM3 DRAM With a TSV Auto-Calibration Scheme and Machine-Learning-Based Layout Optimization," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 1, 2022.
- [55] J. Kim, M. Sullivan, S. Lym, and M. Erez, "All-Inclusive ECC: Thorough End-to-End Protection for Reliable Computer Memory," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2016.
- [56] H. Wang, Y. Li, X. Zhang, X. Zhao, H. Sun, and T. Zhang, "On the Use of DRAM with Unrepaired Weak Cells in Computing Systems," in *Proceedings of the Second International Symposium on Memory Systems*, 2016.
- [57] W. W. Peterson and D. T. Brown, "Cyclic Codes for Error Detection," *Proceedings of the IRE*, vol. 49, no. 1, 1961.
- [58] R. Wu, S. Zhou, J. Lu, Z. Shen, Z. Xu, J. Shu, K. Yang, F. Lin, and Y. Zhang, "Removing Obstacles before Breaking Through the Memory Wall: A Close Look at HBM Errors in the Field," in *Proceedings of the USENIX Annual Technical Conference (USENIX)*, 2024.
- [59] X. Du and C. Li, "Predicting Uncorrectable Memory Errors from the Correctable Error History: No Free Predictors in the Field," in *Proceedings of the International Symposium on Memory Systems*, 2021.
- [60] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The Llama 3 Herd of Models," *arXiv preprint arXiv:2407.21783*, 2024.
- [61] J. Meza, Q. Wu, S. Kumar, and O. Mutlu, "Revisiting Memory Errors in Large-Scale Production Data Centers: Analysis and Modeling of New Trends from the Field," in *Proceedings of the International Conference on Dependable Systems and Networks (DSN)*, 2015.
- [62] M. Nicolaidis, "Design for Soft Error Mitigation," *IEEE Transactions on Device and Materials Reliability*, vol. 5, no. 3, 2005.
- [63] M. V. Beigi, Y. Cao, G. Tsai, S. Gurumurthi, and V. Sridharan, "DDR5 DRAM Faults in the Field," in *Proceedings of the International Conference on Dependable Systems and Networks-Supplemental Volume (DSN-S)*, 2025.
- [64] H. Jeon, G. H. Loh, and M. Annavaram, "Efficient RAS Support for Die-stacked DRAM," in *Proceedings of the International Test Conference (ITC)*, 2014.
- [65] P. J. Nair, V. Sridharan, and M. K. Qureshi, "XED: Exposing On-Die Error Detection Information for Strong Memory Reliability," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2016.
- [66] J. B. Halbert, K. S. Bains, and K. E. Criss, "Memory device on-die error checking and correcting code," US Patent, Nov. 2017, issued Nov. 14, 2017. [Online]. Available: <https://patents.google.com/patent/US9817714B2/en>
- [67] JEDEC standard, "High Bandwidth Memory DRAM (HBM3)," in *JESD238*, 2022.
- [68] S.-L. Gong, J. Kim, S. Lym, M. Sullivan, H. David, and M. Erez, "DUO: Exposing On-Chip Redundancy to Rank-Level ECC for High Reliability," in *Proceedings of the International Symposium on High Performance Computer Architecture (HPCA)*, 2018.
- [69] G. Jung, H. J. Na, S.-H. Kim, and J. Kim, "Dual-Axis ECC: Vertical and Horizontal Error Correction for Storage and Transfer Errors," in *Proceedings of the International Conference on Computer Design (ICCD)*, 2024.
- [70] M. Patel, G. F. de Oliveira, and O. Mutlu, "HARP: Practically and Effectively Identifying Uncorrectable Errors in Memory Chips that Use On-Die Error-Correcting Codes," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2021.
- [71] JEDEC standard, "Low Power Double Data Rate (LPDDR) 5/5X," in *JESD209-5C*, 2023.
- [72] D. Sarwate and N. Shanbhag, "High-Speed Architectures for Reed-Solomon Decoders," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 9, no. 5, 2001.
- [73] L. Saiz, J. Gracia, D. Gil, J.-C. Baraza-Calvo, and P. Gil-Vicente, "Reducing the Overhead of BCH Codes: New Double Error Correction Codes," *Electronics*, vol. 9, 2020.
- [74] V. Sridharan, N. DeBardeleben, S. Blanchard, K. B. Ferreira, J. Stearley, J. Shalf, and S. Gurumurthi, "Memory Errors in Modern Systems: The Good, The Bad, and The Ugly," *ACM SIGARCH Computer Architecture News*, vol. 50, no. 4, 2015.
- [75] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-Sim: An Extensible Simulation Framework for Validated GPU Modeling," in *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020.
- [76] S. Che, J. W. Sheaffer, M. Boyer, L. G. Szafaryn, L. Wang, and K. Skadron, "A Characterization of the Rodinia Benchmark Suite with Comparison to Contemporary CMP Workloads," in *Proceedings of the International Symposium on Workload Characterization (IISWC)*, 2010.
- [77] J. A. Stratton, C. I. Rodrigues, I.-J. Sung, N. Obeid, L.-W. Chang, N. Anssari, G. Liu, and W. mei W. Hwu, "Parboil: A Revised Benchmark Suite for Scientific and Commercial Throughput Computing," *Center for Reliable and High-Performance Computing*, vol. 127, no. 7.2, 2012.
- [78] L. Nai, Y. Xia, I. G. Tanase, H. Kim, and C.-Y. Lin, "GraphBIG: Understanding Graph Computing in the Context of Industrial Solutions," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC)*, 2015.
- [79] M. A. Abella-González, P. Carollo-Fernández, L.-N. Pouchet, F. Rastello, and G. Rodríguez, "PolyBench/Python: Benchmarking Python Environments with Polyhedral Optimizations," in *Proceedings of the 30th ACM SIGPLAN International Conference on Compiler Construction*, 2021.
- [80] S. Cha, O. Seongil, H. Shin, S. Hwang, K. Park, S. J. Jang, J. S. Choi, G. Y. Jin, Y. H. Son, H. Cho, J. H. Ahn, and N. S. Kim, "Defect Analysis and Cost-Effective Resilience Architecture for Future DRAM Devices," in *Proceedings of the International Symposium on High Performance Computer Architecture (HPCA)*, 2017.
- [81] Samsung Electronics, *16Gb HBM Flashbolt*, 2021.
- [82] Micron, "TN-40-07: Calculating memory power for DDR4 SDRAM," 2018.
- [83] N. Chatterjee, M. O'Connor, D. Lee, D. R. Johnson, S. W. Keckler, M. Rhu, and W. J. Dally, "Architecting an Energy-Efficient DRAM System for GPUs," in *Proceedings of the International Symposium on High Performance Computer Architecture (HPCA)*, 2017.
- [84] S. Yun, K. Kyung, J. Cho, J. Choi, J. Kim, B. Kim, S. Lee, K. Sohn, and J. H. Ahn, "Duplex: A Device for Large Language Models with Mixture of Experts, Grouped Query Attention, and Continuous Batching," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2024.
- [85] A. Jarmusch and S. Chandrasekaran, "Microbenchmarking NVIDIA's Blackwell Architecture: An in-depth Architectural Analysis," 2026. [Online]. Available: <https://arxiv.org/abs/2512.02189>

A. Abstract

This artifact includes the source code, scripts, and detailed instructions required to reproduce the key experimental results presented in the paper.

B. Artifact check-list (meta-information)

1) Reliability:

- **Algorithm:** Monte Carlo error injection to estimate reliability under various error types.
- **Program:** C++ simulation
- **Run-time environment:** Tested on Ubuntu 18.04.6 LTS. Compatible with any Linux system equipped with a C++ compliant compiler.
- **Metrics:** Correctable Error rate (CE), Detected Uncorrectable Error rate (DUE), Silent Data Corruption rate (SDC).
- **Output:** Reliability results (Table II & Table III).
- **Experiments:** Experiments are automated via a Python script. Results are saved per configuration.
- **How much time is needed to complete experiments (approximately)?:** Typically under 1 hour, depending on iteration count and configuration.

2) Performance & Energy Consumption:

- **Algorithm:** Architectural simulation to evaluate performance overheads of various ECC schemes and analytical estimation of DRAM energy consumption.
- **Program:** Accel-Sim for GPU performance simulation; Micron DDR4 power calculator for DRAM energy estimation.
- **Data set:** 16 workloads from Rodinia, Parboil, GraphBIG, and PolyBench.
- **Run-time environment:** Ubuntu 18.04.6 LTS with Docker for simulation execution and result post-processing.
- **Metrics:** IPC (normalized to HBM4) and DRAM energy consumption (normalized to HBM4).
- **Output:** IPC results (Fig. 7a) & Energy consumption (Fig. 7b)
- **Experiments:** Experiments are automated via a shell script. Results are saved per configuration.
- **How much time is needed to complete experiments (approximately)?:** Typically under 15 hours, depending on the number of workloads and the simulated input sizes.

C. Description

1) **How to access:** The source code, scripts, and guidelines are publicly available on GitHub (<https://github.com/99june18/Cerberus>)

2) Hardware dependencies:

- **Reliability**
 - No specific hardware is required. The simulation can run on a single-core CPU, but we recommend using a multi-core system to efficiently execute multiple error injection scenarios in parallel.
- **Performance & Energy Consumption**
 - We recommend a multi-core Linux server/workstation capable of running Docker-based cycle-level GPU simulations.

3) Software dependencies:

- **Reliability**
 - Python 3
 - gcc/g++ (or any C++ compliant compiler)
- **Performance & Energy Consumption**
 - Ubuntu 18.04.6 LTS
 - Docker
 - Accel-Sim
 - Python 3

4) **Data sets:** We use 16 workloads drawn from Rodinia, Parboil, GraphBIG, and PolyBench

D. Installation

All source code, scripts, and synthesis scripts will be provided through the public artifact repository on GitHub (<https://github.com/99june18/Cerberus>)

E. Experiment workflow

1) **Reliability:** The artifact repository provides the workflow for reproducing the reliability results in Table II and Table III. The evaluation is based on Monte Carlo error injection under single-location and multi-location fault scenarios, and reports the probabilities of correctable errors (CE), detected uncorrectable errors (DUE), and silent data corruptions (SDC) for each ECC configuration.

2) **Performance & Energy Consumption:** The artifact repository provides a Docker-based workflow for reproducing the GPU performance and DRAM energy results (Fig. 7a & Fig. 7b). The workflow runs Accel-Sim with the baseline HBM4 configuration, applies per-ECC timing changes, executes the 16 benchmark workloads, and post-processes IPC and energy results normalized to HBM4. The energy evaluation uses the operating currents in Table V together with the Micron DDR4 power calculator.

F. Evaluation and expected results

1) **Reliability:** The reliability evaluation reports CE, DUE, and SDC for all evaluated ECC configurations under the fault scenarios in the paper. The aggregated results reproduce Table II and Table III.

2) **Performance & Energy Consumption:** The performance and energy evaluation reproduces Fig. 7a & Fig. 7b by reporting IPC and DRAM energy normalized to HBM4 for Unity ECC, DUO, Cerberus (32b), and Cerberus (40b).